Лабараторная работа 1: Цветовые модели

Цель

Целью лабораторной работы является изучение принципов представления цвета с помощью различных цветовых моделей и разработка интерактивного веб-приложения для преобразования цвета между моделями RGB, CMYK и HLS.

В процессе работы необходимо было не только реализовать математические формулы преобразования, но и связать их с пользовательским интерфейсом. Пользователь должен иметь возможность изменять отдельные компоненты цвета, наблюдать изменение итогового цвета и получать автоматически пересчитанные значения в других цветовых моделях.

Также целью работы являлось получение практического опыта разработки приложения на TypeScript, организации структуры проекта, создания элементов управления и проверки корректности математических преобразований.

Задачи

1. Разобраться, как работает каждая цветовая модель и как изменение её компонентов влияет на итоговый цвет и связано с другими цветовыми моделями.
2. Построить математическую модель преобразования цветовых значений между RGB, CMYK и HLS, включая обработку диапазонов значений, округление и особые случаи.
3. Построить кодовую базу приложения, реализовав типы данных, функции преобразования, хранение текущего цвета и автоматическое обновление значений.
4. Разработать пользовательский интерфейс с числовыми полями, ползунками, цветовой палитрой, HEX-представлением и отображением текущего цвета.

Ход работы

1. Изучение цветовых моделей

На первом этапе были изучены назначение и компоненты используемых цветовых моделей.

RGB применяется для отображения цвета на экранах и состоит из трёх компонентов:

R — красный
G — зелёный
B — синий

Каждая компонента принимает значение от 0 до 255. Чем больше значение компоненты, тем выше интенсивность соответствующего цвета.

CMYK используется преимущественно в полиграфии и состоит из компонентов:

C — голубой
M — пурпурный
Y — жёлтый
K — чёрный

Значения CMYK задаются в процентах от 0 до 100. В этой модели компоненты описывают количество краски.

HLS состоит из:

H — тон
L — светлота
S — насыщенность

Тон задаётся в градусах от 0 до 360, а светлота и насыщенность — в процентах.

Изучение моделей было необходимо для того, чтобы понимать не только формулы, но и смысл изменения каждого значения. Например, изменение H меняет оттенок, изменение L делает цвет светлее или темнее, а изменение S приближает цвет к серому или делает его более насыщенным.

2. Выбор внутреннего представления цвета

Для упрощения логики приложения было выбрано единое внутреннее представление цвета в формате RGB.

В программе текущий цвет хранится в переменной:

```
let rgb: RGB = { r: 103, g: 80, b: 164 };
```

RGB используется как центральная модель, потому что она непосредственно связана с отображением цвета на экране.

При изменении CMYK или HLS программа сначала переводит новое значение в RGB. После этого из полученного RGB рассчитываются остальные модели.

Общий принцип работы выглядит следующим образом:

изменение RGB
→ пересчёт CMYK и HLS

изменение CMYK
→ преобразование в RGB
→ пересчёт HLS и HEX

изменение HLS
→ преобразование в RGB
→ пересчёт CMYK и HEX

Такой подход позволяет хранить только одно основное состояние цвета и не допускать рассинхронизации между моделями.

3. Построение математической модели

Для выполнения преобразований значения RGB сначала нормализуются. Каждая компонента переводится из диапазона 0–255 в диапазон 0–1:

$$r = R / 255$$
$$g = G / 255$$
$$b = B / 255$$

Для преобразования RGB в CMYK сначала определяется чёрная компонента:

$$K = 1 - \max(r, g, b)$$

Затем рассчитываются компоненты C, M и Y:

$$C = (1 - r - K) / (1 - K)$$
$$M = (1 - g - K) / (1 - K)$$
$$Y = (1 - b - K) / (1 - K)$$

После этого значения переводятся в проценты.

Для обратного преобразования используются формулы:

$$R = 255 \times (1 - C) \times (1 - K)$$
$$G = 255 \times (1 - M) \times (1 - K)$$
$$B = 255 \times (1 - Y) \times (1 - K)$$

При преобразовании RGB в HLS сначала находятся максимальная и минимальная компоненты:

$$max = \max(r, g, b)$$
$$min = \min(r, g, b)$$
$$delta = max - min$$

Светлота рассчитывается как:

$$L = (max + min) / 2$$

Тон определяется в зависимости от того, какая RGB-компонента является максимальной. Насыщенность рассчитывается относительно светлоты.

При обратном преобразовании HLS в RGB цветовой круг разбивается на шесть секторов. В зависимости от значения H определяется соотношение красной, зелёной и синей компонент, после чего учитываются светлота и насыщенность.

4. Создание структуры проекта

Проект был разделён на отдельные файлы:

```
Lab_1/
├── index.html
├── package.json
├── src/
│   ├── main.ts
│   ├── color.ts
│   ├── styles.css
│   └── color.test.ts
├── tsconfig.json
└── vite.config.ts
```

Разделение файлов позволяет отделить математические функции от интерфейса и оформления.

5. Реализация функций преобразования

В файле color.ts были описаны типы данных:

```
export type RGB = {
  r: number;
  g: number;
  b: number;
};

export type CMYK = {
  c: number;
  m: number;
  y: number;
  k: number;
};

export type HLS = {
  h: number;
  l: number;
  s: number;
};
```

Также были реализованы вспомогательные функции.

Функция clamp() ограничивает значение допустимым диапазоном:

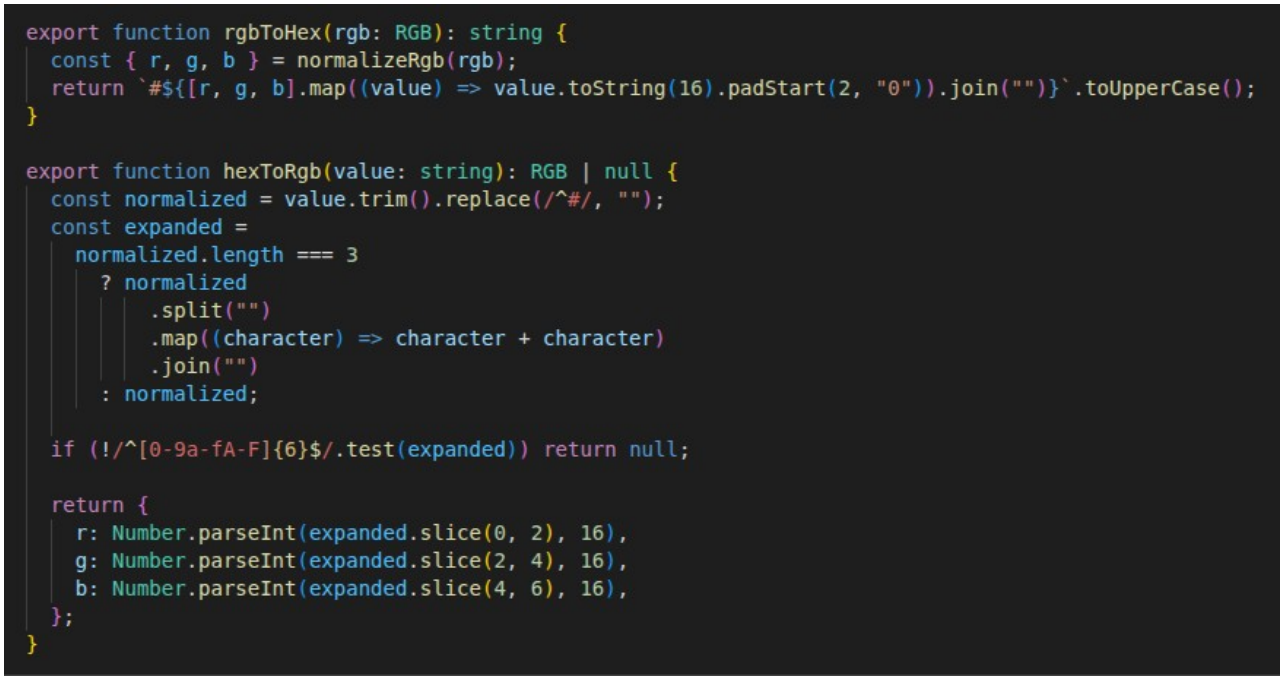
```
export const clamp = (
  value: number,
  min: number,
  max: number
): number =>
  Math.min(max, Math.max(min, value));
```

Функция normalizeRgb() ограничивает RGB значениями от 0 до 255 и округляет их:

```
export const normalizeRgb = ({ r, g, b }: RGB): RGB => ({
  r: Math.round(clamp(r, 0, 255)),
  g: Math.round(clamp(g, 0, 255)),
  b: Math.round(clamp(b, 0, 255)),
});
```

Для преобразования в HEX были реализованы функции:

rgbToHex()
hexToRgb()



```
export function rgbToHex(rgb: RGB): string {
  const { r, g, b } = normalizeRgb(rgb);
  return `#${[r, g, b].map((value) => value.toString(16).padStart(2, "0")).join("")}`.toUpperCase();
}

export function hexToRgb(value: string): RGB | null {
  const normalized = value.trim().replace(/^#/, "");
  const expanded =
    normalized.length === 3
      ? normalized
          .split("")
          .map((character) => character + character)
          .join("")
      : normalized;

  if (!/^[0-9a-fA-F]{6}$/.test(expanded)) return null;

  return {
    r: Number.parseInt(expanded.slice(0, 2), 16),
    g: Number.parseInt(expanded.slice(2, 4), 16),
    b: Number.parseInt(expanded.slice(4, 6), 16),
  };
}
```

Для цветовых моделей:

rgbToCmyk(),
cmykToRgb()

```
export function rgbToCmyk(rgb: RGB): CMYK {
  const { r, g, b } = normalizeRgb(rgb);
  const red = r / 255;
  const green = g / 255;
  const blue = b / 255;
  const black = 1 - Math.max(red, green, blue);

  if (black >= 1) return { c: 0, m: 0, y: 0, k: 100 };

  return {
    c: ((1 - red - black) / (1 - black)) * 100,
    m: ((1 - green - black) / (1 - black)) * 100,
    y: ((1 - blue - black) / (1 - black)) * 100,
    k: black * 100,
  };
}

export function cmykToRgb({ c, m, y, k }: CMYK): RGB {
  const cyan = clamp(c, 0, 100) / 100;
  const magenta = clamp(m, 0, 100) / 100;
  const yellow = clamp(y, 0, 100) / 100;
  const black = clamp(k, 0, 100) / 100;
```

rgbToHls()
hlsToRgb()

```ts
export function rgbToHls(rgb: RGB): HLS {
  const { r, g, b } = normalizeRgb(rgb);
  const red = r / 255;
  const green = g / 255;
  const blue = b / 255;
  const max = Math.max(red, green, blue);
  const min = Math.min(red, green, blue);
  const delta = max - min;
  const lightness = (max + min) / 2;

  return {
    h: normalizeHue(rgbToHue(red, green, blue, delta, max)),
    l: lightness * 100,
    s: delta === 0 ? 0 : (delta / (1 - Math.abs(2 * lightness - 1))) * 100,
  };
}

export function hlsToRgb({ h, l, s }: HLS): RGB {
  const lightness = clamp(l, 0, 100) / 100;
  const saturation = clamp(s, 0, 100) / 100;
  const chroma = (1 - Math.abs(2 * lightness - 1)) * saturation;
  return hueToRgb(h, chroma, lightness - chroma / 2);
}
```

6. Создание пользовательского интерфейса

В main.ts был создан интерфейс приложения.

Для каждой модели автоматически создаётся карточка:

${modelCard("rgb")}
${modelCard("cmyk")}
${modelCard("hls")}

Внутри карточек расположены поля компонентов.

Например, для HLS:

H — Тон
L — Светлота
S — Насыщенность

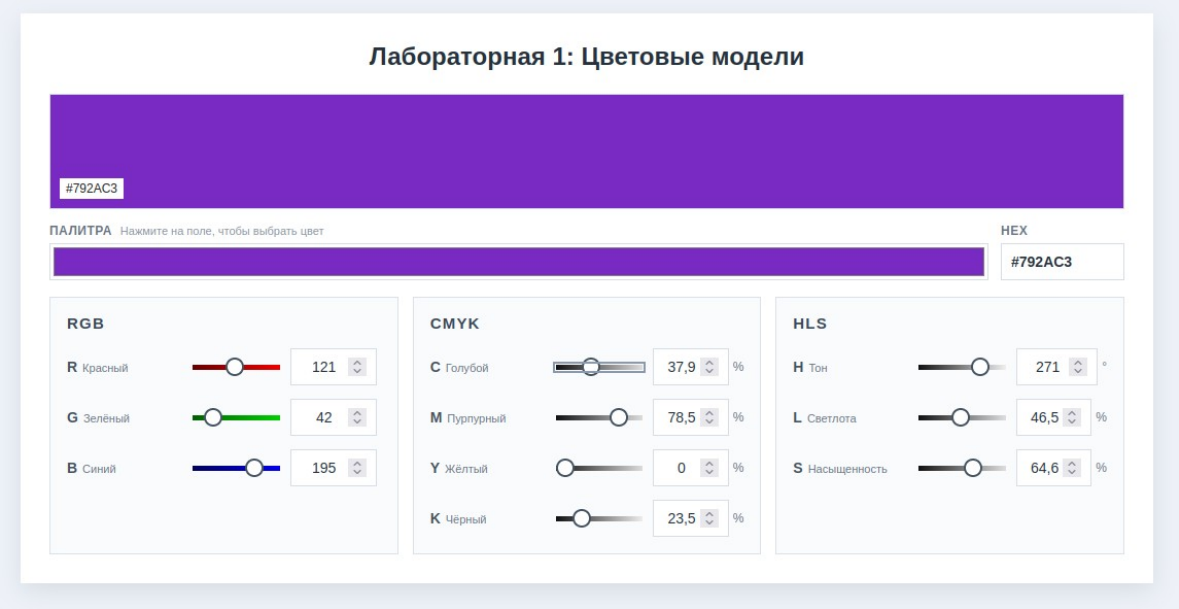
Для каждой компоненты добавлены:

- буквенное обозначение;
- полное название;
- ползунок;
- числовое поле;
- единица измерения.

Кроме карточек моделей были добавлены:

- область предварительного просмотра цвета;
- системная цветовая палитра;

- поле HEX;
- отображение текущего HEX-значения.



7. Реализация изменения значений

Для всех полей и ползунков используется событие input.

Функция bindChannelInputs() определяет, какая модель и какая компонента были изменены:

```
const model = input.dataset.model as Model;
const channel = input.dataset.channel;
```

После изменения ползунка синхронно обновляется числовое поле. Если пользователь меняет числовое поле, обновляется соответствующий ползунок.

Затем вызывается функция:

```
updateFromModel(model);
```

Она считывает значения выбранной модели и выбирает необходимое преобразование:

```
if (model === "rgb") {
  rgb = normalizeRgb(values as RGB);
}

if (model === "cmyk") {
  rgb = cmykToRgb(values as CMYK);
}

if (model === "hls") {
```

```
  rgb = hlsToRgb(values as HLS);
}
```

После получения нового RGB вызывается:

```
render();
```

Функция render() обновляет цвет предварительного просмотра, HEX, значения всех компонентов и положения ползунков.

8. Реализация HEX-ввода

Для HEX был добавлен отдельный обработчик:

```
hexInput.addEventListener(
  "input",
  () => applyHex(hexInput.value)
);
```

Функция applyHex() проверяет корректность значения. Если HEX неправильный, поле помечается как ошибочное:

```
hexInput.setAttribute("aria-invalid", "true");
```

Если значение корректное, оно преобразуется в RGB:

```
const parsed = hexToRgb(value);
rgb = parsed;
render();
```

После этого автоматически обновляются CMYK и HLS.

Результат

В результате разработано интерактивное веб-приложение на TypeScript для работы с цветом.

В приложении реализованы:

- модели RGB, CMYK и HLS;
- преобразования между моделями;
- ввод значений вручную;
- изменение значений с помощью ползунков;
- выбор цвета через системную палитру;
- ввод и отображение HEX;
- автоматический пересчёт всех цветовых моделей;
- видимые подписи всех компонентов;
- предварительный просмотр выбранного цвета;
- проверка математических функций автоматическими тестами;
- сборка проекта с помощью Vite.

Вывод

В ходе работы была освоена связь между различными способами представления цвета. Было установлено, что для синхронизации моделей удобно использовать RGB как центральное внутреннее представление.

Разработанное приложение подтвердило корректность математических преобразований на практике: изменение любого параметра приводит к обновлению текущего цвета и пересчёту остальных моделей. Поставленные задачи выполнены, а полученная структура приложения позволяет в дальнейшем добавлять новые лабораторные работы отдельно от текущего проекта.